

FREE DVD INSIDE PC-BSD 7.0 FIBONACCI EDITION

MAGAZINE

BSD

INS&OUTS

PC-BSD UNCOVERED

EXCLUSIVELY

► Interview with
Matt Olander and Kris Moore

INSIDE

PC-BSD and the PBI System
Personalizing Your PC-BSD Desktop
Using FreeBSD for Off-Site Backups
Building NetBSD for Embedded System Using Cygwin
Open Source Studio to Transmitter Link (OSSTL)
Django FreeBSD

60 PAGES OF PRACTICAL TUTORIALS

Vol.2 No.2 USD14.99
2/2006(4) ISSN 1898-9144



Open Source

Studio to Transmitter Link (OSSTL)

Jason Ellison

A local nonprofit radio station owns a studio that provides feeds for two AM radio stations and one FM radio station. The content provided for the two AM station's is mostly syndicated talk radio with a very little music content.

While the FM radio station's programming is a wide range of audio from talk radio to modern pop music. The studio sends the audio to two remote locations for transmission. The two AM station's mono audio were sent to a transmitter six miles away and the FM station's stereo audio was sent to a transmitter 15 miles away. This was accomplished using multiple microwave links in the 950MHz frequency range. These systems that are used to transmit the audio from the studio to the actual transmitter site are referred to as Studio to Transmitter Links, or STL's for short.

The studio was about to change locations and they were reconsidering the way they transported audio from the studio to the transmitter sites. The current system of using microwave STL links had some drawbacks. The microwave links required annual licensing fees for the 900MHz frequencies used by the system. The particular microwave based STL that the station used were older Marti STL-10's. Due to the age of the equipment in use, audio quality was slightly less than that of some of the competition. This is a major concern for radio stations because sound quality can have a huge impact on retaining listenership. The equipment also could not do some of the more modern tasks such as encoding the the sound into digital formats for instance, streaming to web listeners or archiving for future listeners.

Initial Discussion

I was approached about the possible use of open source software and commodity hardware to replace the current proprietary microwave based STL system. Knowing that moving audio from one location to another was completely in the realm of modern computers, I felt this would be an interesting project that on the surface seemed very doable. I had been playing with low power FM transmitters for a while and had previously considered the need for a low cost and

flexible solution. Once you were in the realm of commodity hardware and open source software, although not trivial, adding functionality becomes a much easier task when compared to proprietary solutions.

Project requirements

The first step was to solidify the project requirements and get them down on paper. After a few initial discussions we were able to get the project requirements clearly defined. The requirements were as follows:

- Move CD quality (or better) audio across an Ipv4 network
- Maintain a constant system latency of less than 2 seconds (from audio input to audio output).
- Deliver the audio stream reliably.
- Provide digital versions of the audio streams for internet radio.
- Transmit the audio using no more than 1.5Mbps (T1)

Research

I scoured the web and newsgroups looking for projects that may accomplish the defined goals. I found a small amount of discussion and even fewer projects. All of the projects that I did find seemed to now be defunct. One such project is flow-stl hosted at sourceforge.net and previously used by Prometheus Radio. I managed to get into contact with Andy Gunn of Prometheus Radio. Unfortunately he informed me that the software was no longer in use and that they were currently using a proprietary software package. I also was unable to obtain a copy of the software they previously used. So it looked like I would not have the benefit of seeing how previous projects attempted this sort of thing.

For the next two months I spent my days and nights at my desk testing various software programs. Each software

package in various configurations. Listening to the audio for changes in latency... making sure that the end result sounded as good as the input. Verifying bandwidth utilization. The whole process very quickly became repetitive. I automated a few of the testing procedures, but it remained a very labor intensive process.

This system had to work over low bandwidth connections. To simulate the expected network conditions, I setup two Cisco 1720 routers with T1 WIC's in a back to back configuration. Thats fancy

speak for saying I used a crossover cable to test the audio transfers across a T1 network. This allowed me to adjust network bandwidth and latency. These were configured at various times with QoS to simulate even lower bandwidth conditions, packet loss and network congestion (Figure 1).

Hardware

The first thing we choose were Audio cards. We needed something of professional quality. For the studio we choose the *M Audio Delta 44*. This card

has four 1/4" TRS balanced analog inputs. For the transmitter side we went with the *M Audio Audiophile 192*, which offered two 1/4" TRS balanced analog outputs. These cards are very high quality, offering frequency responses from 20Hz to 20kHz @ 48kHz. These devices also have very little crosstalk, which means I should be able to use a stereo stream to send two mono audio streams to the AM transmitter site. More over these cards seem to be fairly well supported on open source operating systems either by the operating system itself or through a third

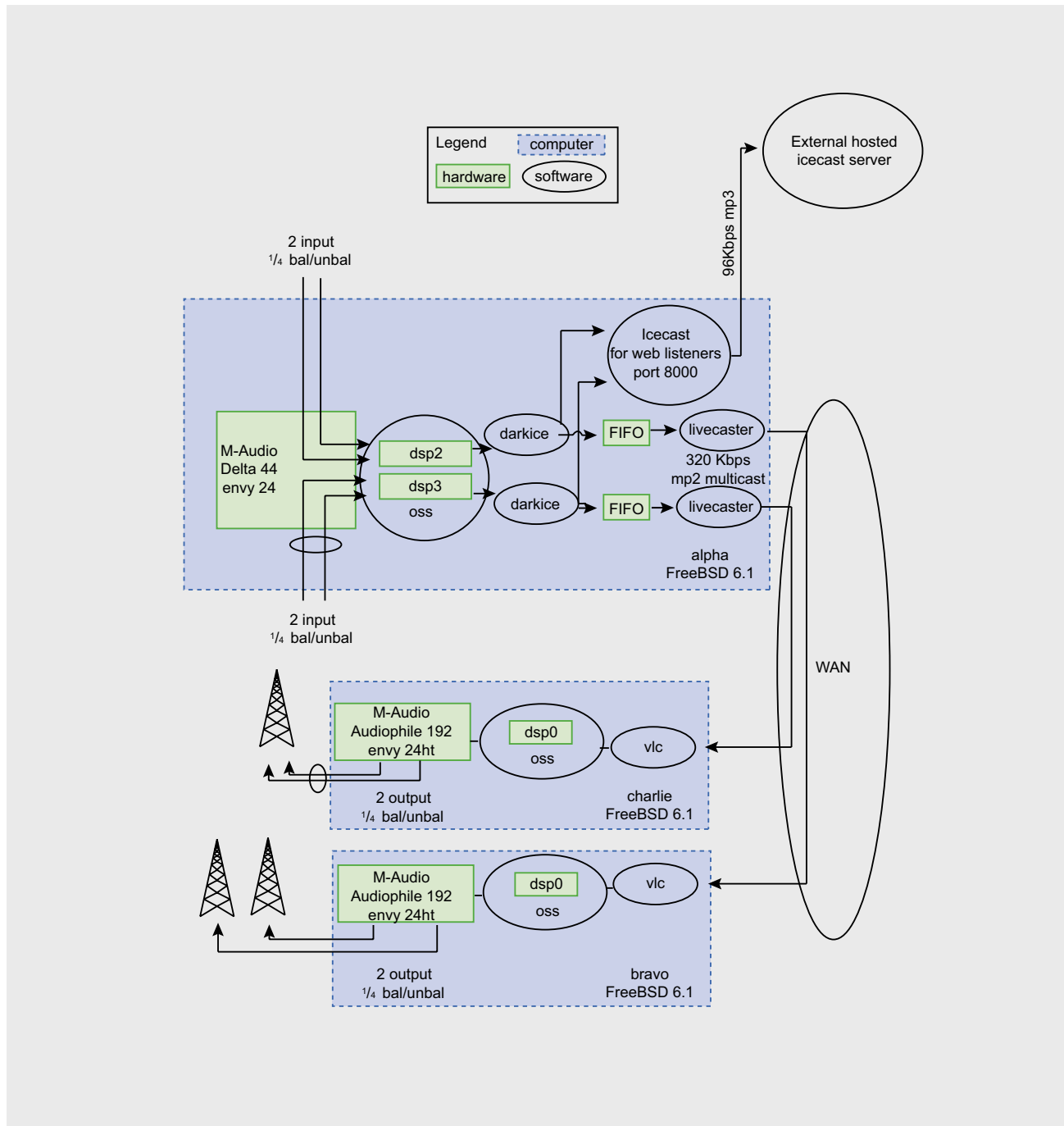


Figure 1. OSSTL Flowchart



party. At the time of the project FreeBSD stable was version 6.1 which did not support these cards. I was able to get support by using 4Front's proprietary OSS drivers.

(since beginning this project, FreeBSD appears to support the M-Audio soundcards via the open source Envy24 and Envy24HT sound drivers) Next were the computers. I was allocated three computers to build the system. The machines were whitebox OEM machines that I assembled to keep cost down. We used basic but reliable components. In the

end, the specs I choose were a little overkill. But I wanted to be on the safe side.

- Studio side (hostname: alpha)
- CPU: AMD Athlon 64 Processor 3500+ (2.1GHz)
- RAM: 1GB of RAM.

In production use, top shows 498MB of memory usage and the cpu between 50% – 60% idle.

- Transmitter side (hostname: bravo and charlie)

- CPU: AMD Sempron Processor 2800+ (1.6GHz)
- RAM: 512MB of RAM.

In production use, top shows 146MB of memory usage and the cpu 99% idle.

Software

For this project I spent countless hours researching many software applications. You could really get lost evaluating all of them. I kept to the projects that had been around for a while and had been well maintained. This was after all going to be used in a live radio station. I needed software that had been in the field for a while.

I settled on Darklce for reading the analog data from the soundcard inputs. Darklce is highly configurable and very stable. It allows for reading from oss interfaces which is exactly what I needed. For output, I do a few things...

To get the audio to the transmitter sites I use a bitrate of 320Kbps. This bitrate was chosen to avoid possible compression artifacts in the audio stream. This is because some of the programming that the station plays has already been compressed. The satellite feed for instance is already digitized at 256Kbps using an MP2 codec. satellite feeds are fairly common these days. During testing I found that when I played previously encoded files with a laptop through the system, I would get fairly severe artifacts unless I used a bitrate that was equal to or above the bitrate of what the original content was digitized at. The audio bound for the transmitter sites is encoded and written to a file which is actually a fifo.

Now that the audio is being fed into a local fifo I needed to get it out to the transmitter sites. This one took a while to figure out... I choose to go with liveCaster which uses the liveMedia library from *live555*. I found a liveCaster binary available from *live555.com*. It is free to use but does not come with the source. The LiveMedia library is the same library that VLC uses to read and write RTSP, RTP and SDP formats. liveCaster can be used to send MPEG audio data in RTP/UDP packets. It has the capability to send in either Unicast or Multicast. This application is rock solid. It does not transcode the audio data so it very fast also.

Listing 1. Compile-Time Configuration Options

```
icecast-2.3.1

no custom configuration

lame-3.96.1

no custom configuration

twolame-0.3.7

no custom configuration

darklce-0.18

./configure --without-alsa --without-jack --without-faac --with-lame
--prefix=/usr/local --with-twolame-prefix=/usr/local

vlc-0.8.5

./configure --enable-oss --enable-twolame --disable-alsa --disable-ffmpeg \
--disable-libmpeg2 --disable-wxwidgets --disable-skins2 \ --enable-
livedotcom --with-livedotcom-tree=/usr/local/src/live
```

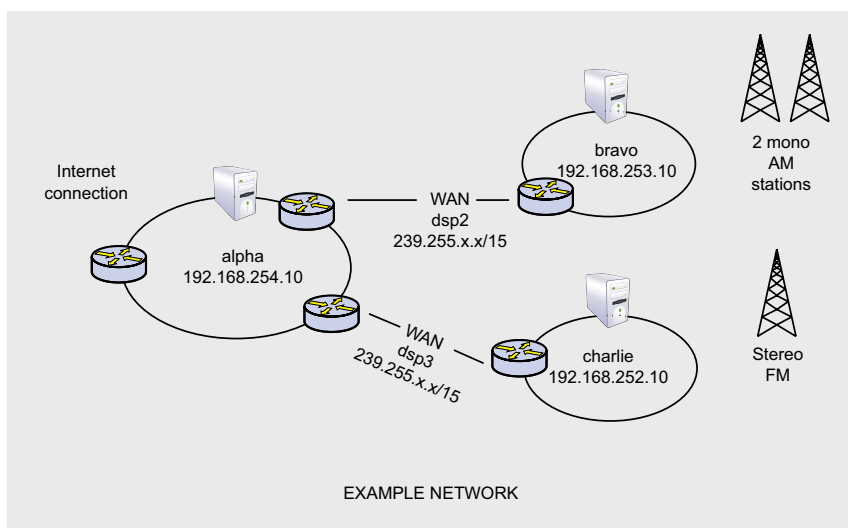


Figure 2. OSSTL Example Network

Listing 2a. Startup Scripts

```

/usr/local/etc/rc.d/darkice-a:

#!/bin/sh
# darkice and livecaster
# PROVIDE: darkice-a
# REQUIRE: DAEMON oss icecast2

. /etc/rc.subr
name="darkice-a"
stop_cmd="darkice_stop"
start_cmd="darkice_start"
pidfile=/var/run/$name

BPS=320
DSP=dsp2

darkice_start(){
    echo -e "\nstarting: $name"
    /usr/local/bin/darkice -v 5 \
    -c /usr/local/etc/dlc/$DSP/$BPS/darkice.cfg &
    echo $! > $pidfile
}

darkice_stop(){
    echo -e "\nstopping: $name"
    kill `cat $pidfile`
    rm -f $pidfile
}

darkice_restart(){
    darkice_stop;
    darkice_start;
}

rc_usage(){
    echo "Usage: $0 [start|stop]"
}

load_rc_config $name
run_rc_command "$1"

/usr/local/etc/rc.d/darkice-b:

#!/bin/sh
# darkice and livecaster
# PROVIDE: darkice-b
# REQUIRE: DAEMON oss icecast2

. /etc/rc.subr
name="darkice-b"
stop_cmd="darkice_stop"
start_cmd="darkice_start"
pidfile=/var/run/$name

BPS=320
DSP=dsp3

darkice_start(){
    echo -e "\nstarting: $name"

    /usr/local/bin/darkice -v 5 \
    -c /usr/local/etc/dlc/$DSP/$BPS/darkice.cfg &
    echo $! > $pidfile
}

darkice_stop(){
    echo -e "\nstopping: $name"
    kill `cat $pidfile`
    rm -f $pidfile
}

darkice_restart(){
    darkice_stop;
    darkice_start;
}

rc_usage(){
    echo "Usage: $0 [start|stop]"
}

load_rc_config $name
run_rc_command "$1"

livecaster_start(){
    #livecaster
    sleep 1
    echo "starting: $name"
    /usr/local/bin/lc \
    -d /usr/local/etc/dlc/$DSP/$BPS/ \
    -s /usr/local/etc/dlc/$DSP/$BPS/stream.fifo &
    echo $! > $pidfile
}

livecaster_stop(){
    echo "stopping: $name"
    kill `cat $pidfile` 2 > /dev/null
    rm -f $pidfile
}

livecaster_restart(){
    livecaster_stop
    sleep 1
    livecaster_start
}

rc_usage(){

```

**Listing 2b.** Startup Scripts & Various Run-Time Configuration Files

```

    echo "Usage: $0 [start|stop]"
}
load_rc_config $name
run_rc_command "$1"

/usr/local/etc/rc.d/livecaster-b:

#!/bin/sh
# livecaster
# PROVIDE: livecaster-b
# REQUIRE: DAEMON oss darkice-b

. /etc/rc.subr

name="livecaster-b"
stop_cmd="livecaster_stop"
start_cmd="livecaster_start"
pidfile=/var/run/$name

BPS=320
DSP=dsp3

livecaster_start(){
    sleep 1
    echo "starting: $name"
    /usr/local/bin/lc \
    -d /usr/local/etc/dlc/$DSP/$BPS/ \
    -s /usr/local/etc/dlc/$DSP/$BPS/stream.fifo &
    echo $! > $pidfile
}
livecaster_stop(){
    echo "stopping: $name"
    kill `cat $pidfile` 2 > /dev/null
    rm -f $pidfile
}
livecaster_restart(){
    livecaster_stop
    sleep 1
    livecaster_start
}
rc_usage(){
    echo "Usage: $0 [start|stop]"
}
load_rc_config $name
run_rc_command "$1"

bravo and charlie

/usr/local/etc/rc.d/020-vlc.sh:

#!/bin/sh
# vlc

echo -n "$0 : "

```

```

start(){
    echo "Start"
    ossmix vmix0-src Production ;
    ossmix envy24.rate 44100 ;
    ossmix envy24.ratelock on ;
    /usr/local/bin/vlc --daemon \
    --no-video \
    --audio --aout oss --aout-rate 44100 --dspdev /dev/
dsp \
    --no-oss-buggy --rt-priority --loop \
    --udp-caching 1000 \
    sdp:///usr/local/etc/livecaster/320bps
}
stop(){
    echo "Stop"
    killall vlc
    sleep 1
}
usage(){
    echo "missing parameter"
    echo "Usage: $0 [start|stop]"
}
case "$1" in
    start)
        start "$2" ;
        ;;
    stop)
        stop ;
        ;;
    *)
        usage ;
        ;;
esac

```

Configuration files:

/usr/local/etc/dlc/dsp3/320/darkice.cfg:

```

[general]
duration = 0 # duration of encoding, in seconds. 0
means forever
bufferSecs = 1 # size of internal slip buffer, in
seconds
reconnect = yes # reconnect to the server(s) if
disconnected
realtime = yes

[input]
device = /dev/dsp3 # OSS DSP soundcard device for the
audio input
sampleRate = 44100 # sample rate in Hz. try 11025,
22050 or 44100
bitsPerSample = 16 # bits per sample. Try 16
channel = 2 # channels. 1 = mono, 2 = stereo

```

Listing 2c. Run-Time Configuration Files

```
[icecast2-0]
bitrateMode = cbr
format = mp3
bitrate = 128
quality = 0.5
server = localhost
port = 8000
password = s3cr3t
mountPoint = lpfm-128.mp3
name = LPFM - Radio With A Vision
description = (C)2007 Simmon and Grundy
url = http://www.example.org/
genre = Other
public = yes

[icecast2-1]
bitrateMode = cbr
format = mp3
bitrate = 96
quality = 0.5
server = localhost
port = 8000
password = s3cr3t
mountPoint = lpfm-96.mp3
name = LPFM - Radio With A Vision
description = (C)2007 Simmon and Grundy
url = http://www.example.org/
genre = Other
public = yes

[icecast2-2]
bitrateMode = cbr
format = mp3
bitrate = 64
quality = 0.5
server = localhost
port = 8000
password = s3cr3t
mountPoint = lpfm-64.mp3
name = LPFM - Radio With A Vision
description = (C)2007 Simmon and Grundy
url = http://www.example.org/
genre = Other
public = yes

[icecast2-3]
bitrateMode = cbr
format = mp3
bitrate = 32
quality = 0.5
server = localhost
port = 8000
password = s3cr3t
mountPoint = lpfm-32.mp3
name = LPFM - Radio With A Vision
description = (C)2007 Simmon and Grundy

url = http://www.example.org/
genre = Other
public = yes

[file-0]
bitrateMode = cbr
format = mp2
bitrate = 320
quality = 0.8
fileName = /usr/local/etc/dlc/dsp3/320/stream.fifo
samplerate = 44100
highpass = 25000
lowpass = 10

/usr/local/etc/dlc/dsp2/320/darkice.cfg:

[general]
duration = 0 # duration of encoding, in seconds. 0
means forever
bufferSecs = 1 # size of internal slip buffer, in
seconds
reconnect = yes # reconnect to the server(s) if
disconnected
realtime = yes

[input]
device = /dev/dsp2 # OSS DSP soundcard device for the
audio input
sampleRate = 44100 # sample rate in Hz. try 11025,
22050 or 44100
bitsPerSample = 16 # bits per sample. Try 16
channel = 2 # channels. 1 = mono, 2 = stereo

[file-0]
bitrateMode = cbr
format = mp2
bitrate = 320
quality = 0.8
fileName = /usr/local/etc/dlc/dsp2/320/stream.fifo
samplerate = 44100
highpass = 25000
lowpass = 10

/usr/local/etc/icecast.xml:

<icecast>
<limits>
  <clients>10</clients>
  <sources>5</sources>
  <threadpool>5</threadpool>
  <queue-size>524288</queue-size>
  <client-timeout>30</client-timeout>
  <header-timeout>15</header-timeout>
  <source-timeout>10</source-timeout>
  <burst-on-connect>1</burst-on-connect>
  <burst-size>65535</burst-size>
</limits>
```



For the web listeners DarkIce encodes the audio into four different bitrates: 128Kbps, 96Kbps, 64Kbps and 32Kbps. These lower bitrates are all that are needed for delivery to a remote IceCast server. In fact the radio station is only using the 96Kbps and 32Kbps streams. The remote IceCast server is hosted on a well connected network so it can easily stream the data to a larger number of web listeners.

IceCast was implemented to stream music for internet radio listeners. The web listeners connect to an IceCast server hosted by a third party on a network with a bit more bandwidth. Originally the data was sent directly to that remote IceCast server from DarkIce. Unfortunately, we noticed during testing that a failure of the remote IceCast server can cause your local DarkIce server to crash. Adding a local IceCast

server in the chain resolved these concerns.

At the actual transmitter sites I used VLC player to read the audio data from the network. VLC then outputs the sound via the M Audio Audiophile 192. And again, it uses the same liveMedia library to decode the network stream as we used to create it. This player just works. It uses minimum resources and can be run from the command line.

A few other utilities I found invaluable were sox and festival. I used sox to generate test frequencies to check that the needed frequencies were traversing the entire system untouched by filters. Festival was used to generate human voices into wav files. I would have festival say, *1 second tone at 10KHz...* and then use sox to play a 10KHz sound for one second.

Then I would play other frequencies for different periods. This was tremendously helpful in tweaking the setup (Figure 2).

Details

For installing FreeBSD 6.x OS, I followed the *installing FreeBSD* section of the FreeBSD Handbook (which is available online of course). One thing that I would have liked to have known from the beginning was the expected disk usage. In the end total disk usage was right at 2GB. The included all of the sources. The following is what disk usage looks like on alpha, which was used to build all the software.

```
alpha# df -h
Filesystem      Size      Used Avail Capacity
Mounted on
/dev/ad4s1a 989M      91M 819M   10% /
/dev/ad4s1e 989M    1.4M 909M    0% /tmp
/dev/ad4s1f 68G     1.6G 61G     3% /usr
/dev/ad4s1d 989M      72M 838M    8% /var
```

Listing 2d. IceCast Run-Time Configuration Files

```
<authentication>
  <source-password>s3cr3t</source-password>
  <relay-password>s3cr3t</relay-password>
  <admin-user>admin</admin-user>
  <admin-password>hackme</admin-password>
</authentication>
<hostname>alpha</hostname>
<listen-socket>
<port>8000</port>
</listen-socket>
<fileserve>1</fileserve>
<paths>
  <basedir>/usr/local/share/icecast</basedir>
  <logdir>/usr/local/var/log/icecast</logdir>
  <webroot>/usr/local/share/icecast/web</webroot>
  <adminroot>/usr/local/share/icecast/admin</adminroot>
  <alias source="/" dest="/status.xml"/>
</paths>
<logging>
  <accesslog>access.log</accesslog>
  <errorlog>error.log</errorlog>
  <loglevel>3</loglevel> <!-- 4 Debug, 3 Info, 2 Warn, 1 Error -->
  <logsize>10000</logsize> <!-- Max size of a logfile -->
</logging>
<security>
  <chroot>0</chroot>
  <changeowner>
    <user>nobody</user>
    <group>nogroup</group>
  </changeowner>
</security>
</icecast>
```

Building the software

Most of the software used in this project was compiled using original sources. This resulting system has proven to be extremely reliable. This says a lot about the quality of code. The fact that software is constantly evolving can not be denied. As the complexity of a project grows due to new features and enhancements: the chance for bugs to go unnoticed increases. Because of this it is important to noted exactly which versions of the relevant software packages I am using. Being that the software was downloaded and compiled from source, I also note which options the sources were compiled with (Listing 1).

Custom start up scripts

Custom startup scripts were created to control the various software components. I took advantage of the dependencies to control the start order of the services on the server at the studio. The desired order of start would be oss, IceCast, DarkIce, liveCaster. One DarkIce and one liveCaster process need to be started per soundcard (Listing 2).

The liveCaster GUI was used to generate the *SELF.txt* file. This file is used to configure the liveCaster

daemon. It was also used to generate the sdp file used to instruct VLC what audio to play from the multicast streams (Listing 3).

VLC's SDP files

VLC's knows which audio stream to play because of the information contained in the SDP file. The SDP file used by VLC matches the parameters found in the `SELFTXT` file that configures liveCaster. Since we have two different streams being sent, we have two different SDP files. One of the SDP files describes the stereo FM audio stream. While the other SDP file is used describes the two AM mono signals that are being streamed in the individual channels of the MP2 stereo signal (Listing 4).

Web interface

A simple web interface was created at all three locations that allows the operator to start and stop all daemons. It also allows you to view the current audio levels. This is very useful when trouble shooting a problem. I used PHP and bash/awk scripting to get the volume levels from the `ossmix` program output. With some Ajax I was able to display the audio levels using images. The result was similar to the lights you find on high end audio equipment.

Expansion

One of the things I wanted most for this project is add in multicast output directly into Darklce. Even though liveCaster is available free, Ross Finlayson of live555 has not release the source code for the application. After posting my results to relevant mailing lists, I received a large amount of interest from people all over over the world. I believe that all portions of this setup should be open source. The availability of the source code would allow the community to make any needed future changes. Adding multicast output to Darklce requires writing an additional `Sink` for Darklce using the liveMedia library for encoding the data into RTP/UDP. This was something that I began work on but during testing liveCaster worked so well it became less important than some of the other issues the client had. After the project was proven stable for a month the system was put into production. Being that the system was in

If you wish to contribute to BSD magazine, share your knowledge and skills with other BSD users - do not hesitate - read the guidelines on our website and email us your idea for an article.

Join our team!

**Become BSD magazine
Author or Betatester**

As a betatester you can decide on the contents and the form of our quarterly. It can be you who read the articles before everybody else and suggest the changes to the author.

**Contact us:
editors@bsdmag.org
www.bsdmag.org**

**Listing 3.** liveCaster Run-Time Configuration Files

```

/usr/local/etc/dlc/dsp2/320/SELF.txt:

info dsp2-320bps MP2 audio
bwLimit 320
progId P:10.0.0.109:1172752654
nickname dsp2-320bps
outputMode audio
SDPdir
GroupEId 239.255.7.113 54010 {15 nokey}
tunnelState 0 {} 10100 10100 1

/usr/local/etc/dlc/dsp3/320/SELF.txt:

bwLimit 320
info dsp3-320bps MP2 audio
progId P:10.0.0.109:1172752749
SDPdir local-default
outputMode audio
nickname

dsp3-320bps
tunnelState 0 {} 10100 10100 1
GroupEId 239.255.149.81 56910 {15 nokey}

```

Listing 4. SDP files used by VLC

```

Bravo:

/usr/local/etc/livecaster/320bps/dsp2-320bps.sdp:

v=0
o=- 1172752654 3381741506 IN IP4 10.0.0.109
s=dsp2-320bps
i=dsp2-320bps MP2 audio
b=AS:320
t=3381741506 3381743126
a=type:broadcast
a=tool:liveCaster
m=audio 54010 RTP/AVP 14
c=IN IP4 239.255.7.113/15
charlie:

/usr/local/etc/livecaster/320bps/dsp3-320bps.sdp:

v=0
o=- 1172752749 3381741714 IN IP4 10.0.0.109
s=dsp3-320bps
i=dsp3-320bps MP2 audio
b=AS:320
t=3381743230 3381744850
a=type:broadcast
a=tool:liveCaster
m=audio 56910 RTP/AVP 14
c=IN IP4 239.255.149.81/15

```

production, I wasn't real keen on making major changes.

There were some unknowns about the hardware requirements when I started this. I know now that this system could operate with less powerful hardware than what was used. Validating performance on various hardware platforms with various codecs and bitrates would be very valuable. I would especially like to validate small form factor devices. An environmentally hardened device with a watchdog timer would be ideal for the transmitter sites. With a watchdog timer on the device the system would reboot if it were to lockup... This would save a trip to the transmitter for the radio engineer.

Conclusion

This was a very successful and exciting project. I met and exceeded all of the objectives laid out by the client. The quality of the radio stations audio broadcast has increased dramatically. After switching to this system, the radio station sounds as good as or better than any radio station on the dial. The reliability this system has increased. The ability to readily stream to web listeners on-line was an exciting feature for the station operators. This allowed them to reach a wider audience and offer the service in more convenient ways.

When I completed this project I emailed information to darkice, freebsd, and vlc mailing lists. The Community response I recieved was positive. I had people write from all over the world requesting more information. In the future I will probably start a website dedicated to OSSTL. This website will host all the documentation needed to recreate the system from scratch. It would also host a mailing list so that people could obtain help and share their experience.



References

- FreeBSD <http://www.freebsd.org/>
- DarkIce <http://darkice.tyrell.hu/>
- liveCaster and liveMedia streaming media library <http://www.live555.com/>
- VLC media player <http://www.videolan.org/vlc/>